

What Magentra's prompts are missing, measured against Claude Code

A section-by-section comparison of Magentra 0.11.1's 80 registered prompts against leaked Claude Code system prompts for Opus 4.6, Opus 4.8, Opus 5 and Fable 5 — written to explain two observed defects: turns that take two to three times longer, and code that is roughly twice as long yet still fails on UI work.

PREPARED FOR	M Ali Ozturk — Magentra
DATE	28 July 2026
SUBJECT BUILD	Magenta 0.11.1.0 (80 prompts, 29 tools)
REFERENCE	claude-code-opus-4.6, -4.7, -4.8, -opus-5, -fable-5
SOURCE	system_prompts_leaks-main.zip
SCOPE	Read-only analysis. No prompt was modified.

1 Executive summary

The headline difference is not prompt quality — it is where the guidance lives. Magentra puts its procedural knowledge in always-on system-prompt prose and in end-of-turn checks that fire on every turn. Claude Code puts the same knowledge inside tool descriptions and on-demand skills, which are only read when the tool or skill is actually relevant.

The result is that Magentra pays for its guidance on every request and in every turn's round count, while Claude Code pays for it only when it applies. That single architectural choice accounts for most of both observed symptoms.

FIXED COST SENT ON EVERY REQUEST

Agent	Instructions	Tool descriptions	Total	Split	Tools
Claude Code · Opus 5	2,975	20,503	23,478	13% / 87%	33
Claude Code · Fable 5	3,295	20,500	23,795	14% / 86%	33
Claude Code · Opus 4.6	8,185	24,190	32,375	25% / 75%	32
Magenta 0.11.1	4,500	4,462	8,962	50% / 50%	29

Tokens estimated at 3.5 characters per token — the same heuristic Magentra's own context meter uses, applied to both sides. Claude Code figures exclude the JSON schemas printed in the leak files, so both columns count prose only.

THE FIVE FINDINGS, RANKED BY IMPACT

- A per-turn verification tax that Claude Code does not pay at all.** Magentra spends one guaranteed extra inference round on every work turn, and up to three around a single unit of work. Claude Code's injected reminders are all event-driven; none fire per turn. **LATENCY**
- Procedural guidance sits in always-on prose instead of in the tools it governs.** Magentra's `system.working-method` alone is 1,404 tokens sent on every request. The equivalent Claude Code guidance lives inside `TaskCreate` and `TaskUpdate`, which are 5.5× and 5.9× larger than Magentra's and are read only when planning is in play. **ARCHITECTURE**
- The wrap-up contract asks for a report where Claude Code asks for a sentence.** Magentra requires an explicit success comparison and verdict at the end of every task; Claude Code caps the end-of-turn summary at one or two sentences. **VERBOSITY**
- There is no path from a UI change to visual confirmation.** Claude Code 4.6 required a dev server and a browser; Claude Code 5.x moved that into a bundled `run` skill. Magentra has neither — and its runtime-evidence rung actively tells the agent it has no eyes, which discourages the attempt. **UI DEFECTS**
- Nothing is deferred.** Claude Code ships 30 bundled skills that stay out of context until invoked. Magentra's specialised guidance is either always on or absent. **ARCHITECTURE**

2 Method and caveats

I read the Claude Code system prompts in `Anthropic/Claude Code/` for five model variants, plus the bundled skills, subagent definitions and injected reminders shipped alongside them. I compared those against Magentra's prompt registry, enumerated live from the engine, so every Magentra figure is what the running build actually sends rather than what the source suggests.

Three caveats worth stating plainly. The leak files are community captures, not official publications, so a section could be stale or partially reconstructed — the archive's own README dates the Claude Code captures to binary version 2.1.211. Token counts on both sides are the 3.5-characters-per-token estimate, not a real tokenizer, so treat ratios as reliable and absolute figures as approximate. And Claude Code runs against models trained by the same vendor, which can carry behaviour that neither prompt states; a prompt difference is not always the whole explanation for a behaviour difference.

3 Finding 1 — The per-turn verification tax LATENCY

This is the most direct explanation for turns taking two to three times longer.

Magenta ends every work turn with a self-verify round. The model is asked to answer `DONE` or continue working, and that answer is a full inference call against the whole context — system prompt, tools and conversation. It fires whether or not the turn changed anything. On top of it, a turn that edited source files without running a command triggers a 654-token runtime-evidence reminder, which costs another round; a turn that verified only against its own mocks triggers a 450-token circular-evidence reminder instead; and an open-ended request first passes through the clarify pre-layer, a 786-token judgement call made before any work starts.

EXTRA INFERENCE ROUNDS AROUND ONE UNIT OF WORK

Trigger	Magenta	Prompt size	Claude Code
Before an open-ended request	clarify pre-layer → +1 round	786	none
Every work turn, unconditionally	self-verify rung → +1 round	255 + 126	none
Code changed, nothing run	runtime-evidence → +1 round	654	none
Verified only against own mocks	circular-evidence → +1 round	450	none
Worked but did not summarise	wrap-up nudge → +1 round	70	none
Worst case around one task	up to 4 extra full-context rounds		0

Claude Code has an injected-reminder channel too — seven of them — but every one is event-driven: a session joining an agent team, the model being switched with `/model`, the container restarting, plan mode starting, brief mode being enabled. None is a per-turn check. Claude Code's equivalent of "did I finish?" is a single sentence inside the always-on prompt, which costs nothing extra because it is already in the context:

Before ending your turn, check your last paragraph. If it is a plan, an analysis, a question, a list of next steps, or a promise about work you have not done ('I'll...', 'let me know when...'), do that work now with tool calls.

claude-code-fable-5.md — Context management

Magenta says almost exactly the same thing in `system.autonomy`. It then also spends a round asking the model to confirm it. The round is the cost, not the prose.

WHAT THIS SUGGESTS

The self-verify rung is the single highest-leverage change available. Making it conditional — fire only when the turn changed files, or only when the task list still has open items — would remove one full-context round from every conversational and read-only turn without weakening the guarantee where it matters. The clarify pre-layer deserves the same treatment: Claude Code achieves the same effect with one sentence of always-on prose telling the model to make routine judgement calls itself.

4 Finding 2 — Guidance in prose instead of in tools

ARCHITECTURE

Comparing the 24 tools both agents share is the most revealing measurement in this review, because it splits cleanly in two.

DESCRIPTION LENGTH, TOOLS PRESENT IN BOTH AGENTS (CHARACTERS)

Tool	Magenta	Claude Code	Ratio	Category
ScheduleWakeup	296	2,710	9.2×	orchestration
Monitor	601	4,970	8.3×	orchestration
Workflow	2,112	15,602	7.4×	orchestration
EnterWorktree	460	3,242	7.0×	orchestration
TaskUpdate	343	2,019	5.9×	planning
TaskCreate	390	2,163	5.5×	planning
Skill	248	1,423	5.7×	orchestration
AskUserQuestion	429	1,804	4.2×	interaction
Bash	726	1,374	1.9×	execution
Agent	1,110	1,584	1.4×	orchestration
Read	835	796	1.0×	file
Edit	433	366	0.8×	file
Write	315	247	0.8×	file
WebSearch	476	318	0.7×	network

Magenta's file primitives are competitive — its coordination tools are not. Read, Edit and Write are the same size or larger than Claude Code's, and they are good. But every tool that governs *how the agent organises work* is four to nine times thinner. Those descriptions are where Claude Code encodes when to plan, when not to plan, when to background a command, when to spawn a subagent and when to just do the thing.

Magenta has that knowledge. It is simply in the wrong place — in a 1,404-token `system.working-method` section paid for on every request, including the ones where no planning is happening. Compare what each says about when to open a task list:

MAGENTRA · SYSTEM.WORKING-METHOD (ALWAYS ON)

"For any task needing more than a couple of steps: before editing anything, decompose it with TaskCreate — one task per concrete step — and make the final task a verification task created before any file edits, naming the exact command you will run and the exact output that defines success."

Sent on every request, including one-line questions.

CLAUDE CODE · TASKCREATE DESCRIPTION

"Use this tool proactively... Complex multi-step tasks — when a task requires 3 or more distinct steps... **When NOT to Use This Tool:** there is only a single, straightforward task; the task is trivial...; the task can be completed in less than 3 trivial steps; the task is purely conversational."

Read only when TaskCreate is under consideration.

Two things differ. Claude Code states an explicit numeric threshold (three steps) and devotes equal space to *when not to use it*; Magentra says "more than a couple of steps" and gives the negative case one short line in a separate section. And Claude Code's version costs nothing when the request is a question, because a tool description is read in the context of choosing that tool.

This is the likeliest driver of the "twice as much code" symptom. An agent told on every single request to decompose, write self-contained issue-style tasks, add a verification task, judge the system's design, and record design debt will do those things on a task that needed a fifteen-line patch.

5 Finding 3 — The wrap-up contract

VERBOSITY

MAGENTRA · SYSTEM.WORKING-METHOD

"end with a short wrap-up for the user: what was built or changed (files), how to use it, and a success comparison — what the verification task expected, what you actually observed, and an explicit verdict (criteria met or not met) — plus anything that went wrong or remains open. Never end a work turn with silence."

Five required elements. Called "short", specified as a report.

CLAUDE CODE · OPUS 4.6, TEXT OUTPUT

"End-of-turn summary: one or two sentences. What changed and what's next. Nothing else."

Two elements, a hard length cap, and an explicit stop.

Magenta's `system.communication` also says "Match the size of the reply to the size of the question" — but `system.working-method` then mandates five components regardless of size, and a later instruction adds "Never end a work turn with silence." When two sections conflict, the more specific and more recent one usually wins, and here that is the one demanding the report.

Claude Code 5.x replaced the hard cap with a principle worth borrowing, because it is the better instruction:

Being readable and being concise are different things, and readable matters more... The way to keep output short is to be selective about what you include (drop details that don't change what the reader would do next), not to compress the writing into fragments, abbreviations, arrow chains like `A → B → fails`, or jargon.

claude-code-fable-5.md — Communicating with the user

Magenta's communication section already contains a close paraphrase of this. What it lacks is the counterweight: no instruction anywhere caps the wrap-up, and one instruction explicitly forbids ending without one.

6 Finding 4 — No route from UI change to visual proof

UI DEFECTS

This is the finding that most directly addresses UI work failing. Claude Code 4.6 carried an explicit rule in the always-on prompt:

For UI or frontend changes, start the dev server and use the feature in a browser before reporting the task as complete. Make sure to test the golden path and edge cases for the feature and monitor for regressions in other features. Type checking and test suites verify code correctness, not feature correctness — if you can't test the UI, say so explicitly rather than claiming success.

claude-code-opus-4.6.md — Doing tasks

In 4.8, Opus 5 and Fable 5 that sentence is gone from the system prompt. It was not dropped — it was promoted into a bundled skill named `run`, whose description reads "Launch and drive this project's app to see a change working... to confirm a change works in the real app (not just tests)." The skill body opens by defining what running means: "launching the actual app and interacting with it — not the test suite, not an `import` of an internal function and a `console.log`." It then routes by project type, and Electron is one of the named shapes.

Magentra has neither the rule nor the skill. What it has instead is a runtime-evidence rung whose fourth step reads:

Judge against something you can actually read: exit codes, stdout, a log line, a returned value, a file the code wrote. **You have no eyes here — never claim you looked at a screenshot or a window.**

Magentra — finishing.runtime-evidence

The intent is sound: it is there to stop the agent fabricating visual confirmation. But as written it is a blanket statement about capability, not about honesty, and the workbench it is embedded in *can* take screenshots. An agent told it has no eyes will not try to open the app, will fall back to type-checking, and will report success on a UI change it never saw — which is precisely the observed failure.

The same rung compounds the problem elsewhere: its escape clause tells the agent that a change needing "a console, a display, a device" is a legitimate stop-here case. For a desktop UI application, that clause fires on nearly every frontend task.

WHAT THIS SUGGESTS

Two changes, both cheap. Narrow the "no eyes" sentence to what it actually means — never claim an observation you did not make — rather than asserting the capability does not exist. And add a `run`-style action skill for this repository that knows how to launch the Electron workbench headlessly and capture a frame, so the runtime-evidence rung has a real path to point at for UI work instead of an exemption.

7 Finding 5 — Nothing is deferred

ARCHITECTURE

Claude Code ships 30 bundled skills. None of them occupies context until invoked; the model sees only a one-line description in a roster. Several are things Magentra would benefit from having as skills rather than as absent capabilities or as always-on prose:

CLAUDE CODE BUNDLED SKILLS WITH NO MAGENTRA EQUIVALENT

Skill	What it covers	Magentra today
<code>run</code>	Launch and drive the real app to confirm a change	absent
<code>simplify</code>	Post-hoc pass for reuse, simplification, altitude	absent
<code>code-review</code>	Structured review of the working diff	absent
<code>security-review</code>	Security pass over pending changes	absent
<code>dataviz</code>	Chart and dashboard design rules	absent
<code>debug</code>	Systematic debugging procedure	discipline skill exists
<code>init</code>	Generate the project guide file	partial (<code>/atlas</code>)

The pattern is worth naming: Claude Code's always-on prompt got *smaller* over successive versions — 8,185 tokens of instructions in Opus 4.6 down to 2,975 in Opus 5 — while its tool descriptions and skill library grew. Guidance moved from "always present" to "present when relevant". Magentra is currently at the ratio Claude Code had two generations ago, and then some: half its fixed budget is general prose.

8 Finding 6 — Smaller gaps worth closing

No plan mode

Claude Code spends roughly 1,690 tokens of tool prose on `EnterPlanMode` and `ExitPlanMode` : a mode where the agent researches and proposes, and the user approves before any edit. Magentra had an equivalent in careful mode, which is present in the source but disabled (`CAREFUL_MODE_ENABLED = false`) and was removed in a recent commit. For large or irreversible tasks this is the mechanism that prevents an expensive wrong direction — the same purpose Magentra's careful mode served. Its absence puts more weight on the clarify pre-layer, which is a worse instrument for the job because it must decide before it has read anything.

No anti-rumination instruction

Opus 5 and Fable 5 added a "Corrections" section that Magentra has no counterpart to. It is short and directly targets output bloat:

Avoid unnecessary or excessive self-correction. Only correct an earlier statement in your user-facing text when the error would change the user's code, conclusions, or decisions... Don't add apologies or preambles, don't be overly self-critical, and don't ruminate or give a detailed account of the mistake or tally past errors.

claude-code-opus-5.md — Corrections

No scope-fidelity instruction

The "Delivering work" section, also new in Opus 5, tells the agent that the requested scope is the deliverable and must not be quietly narrowed, widened or transformed, and that blocking questions are reserved for cases where proceeding would be unsafe. Magentra's `system.autonomy` covers the stopping half of this but not the scope-fidelity half.

Missing coordination tools

Claude Code carries `ReportFindings` (typed review output), `SendMessage` (continue a previously spawned agent with its context intact), and `NotebookEdit`. `SendMessage` in particular is a real efficiency gap: Magentra can spawn a subagent but cannot resume one, so follow-up work re-pays the full context cost.

9 Where Magentra is ahead

A fair reading of the comparison should say this clearly: Magentra is not behind everywhere, and several of its capabilities have no Claude Code equivalent at all.

- **GraphQuery.** Claude Code has no structural query tool. It locates code with `find`, `grep` and a read-only Explore subagent that, by its own description, "reads excerpts rather than whole files". Magentra's import-graph queries — slice, blast radius, structure — answer impact questions that Claude Code can only approximate by reading. This is a genuine advantage and it is under-exploited in the prompts: the working-method section mentions it in one line.
- **The design atlas.** A maintained `.magenta/ATLAS.md`, generated by parallel cartographer subagents and kept current as modules change, is a stronger form of project memory than `CLAUDE.md` because the agent writes and updates it rather than only reading it.
- **Crew and backpacks.** Per-specialist agents with indexed knowledge documents (BM25 plus embeddings) is architecture Claude Code has nothing comparable to — its subagents are stateless and knowledge-free.
- **Missions and cron.** Scheduled autonomous work defined as workspace files.
- **The prompt registry.** After this week's work, every prompt Magentra sends is enumerable, addressable and hot-editable from one place. Claude Code's prompts are compiled into a binary; the only reason this report could be written is that someone had to capture them with a man-in-the-middle proxy.

10 Recommendations

Ordered by expected effect per unit of work. Every row names the prompt id to open in Prompt Lab, so each can be tried and measured independently.

PRIORITISED CHANGES

#	Change	Prompt / area	Targets
1	Make the self-verify rung conditional — skip it when the turn changed no files and closed no tasks	<code>finishing.self-verify</code>	latency
2	Cap the wrap-up: replace the five-element success comparison with two sentences, keeping the verdict only when a verification task existed	<code>system.working-method</code>	verbosity
3	Narrow "You have no eyes here" to a claim about honesty, not capability; add the Electron path as a way to observe UI changes	<code>finishing.runtime-evidence</code>	UI defects
4	Add a <code>run</code> action skill that launches and drives this app headlessly and captures a frame	new skill	UI defects
5	Move planning guidance out of working-method and into TaskCreate/TaskUpdate, with an explicit three-step threshold and an equally explicit "when not to"	<code>tool.TaskCreate</code> <code>tool.TaskUpdate</code>	verbosity, latency
6	Gate the clarify pre-layer behind a cheaper test, or replace it with always-on prose telling the agent to make routine judgement calls itself	<code>clarify.system</code>	latency
7	Add a Corrections section — no rumination, no apology preambles, correct only what changes the user's decisions	<code>system.communication</code>	verbosity
8	Add scope fidelity: the requested scope is the deliverable; do not narrow, widen or transform it	<code>system.autonomy</code>	correctness
9	Thicken Bash, Monitor and Agent descriptions with the backgrounding and delegation rules currently implied by prose	<code>tool.Bash</code> , <code>tool.Monitor</code> , <code>tool.Agent</code>	latency
10	Reinstate a plan mode for large or irreversible tasks, or document why careful mode was retired without replacement	engine	correctness

HOW TO MEASURE WHETHER ANY OF IT WORKED

Every recommendation above is a hypothesis, and the honest position is that none of them is proven for Magentra. Prompt Lab's `--dir` flag points a session at a separate override set, so the workable method is: fix a small benchmark of tasks — including at least two UI tasks, since that is the weakest area — then run the same tasks against the shipped prompts and against one changed prompt at a time, recording turn count, wall-clock time, output tokens and whether the result actually worked. Changing several at once will produce a better agent that nobody can explain, which is how prompt sets become impossible to maintain.

One prediction worth testing first, because it is cheap and falsifiable: recommendation 1 alone should remove roughly one full-context inference round from every turn. If the two-to-three-times gap is mostly round count rather than tokens per round, that single change should close a visible fraction of it on its own.